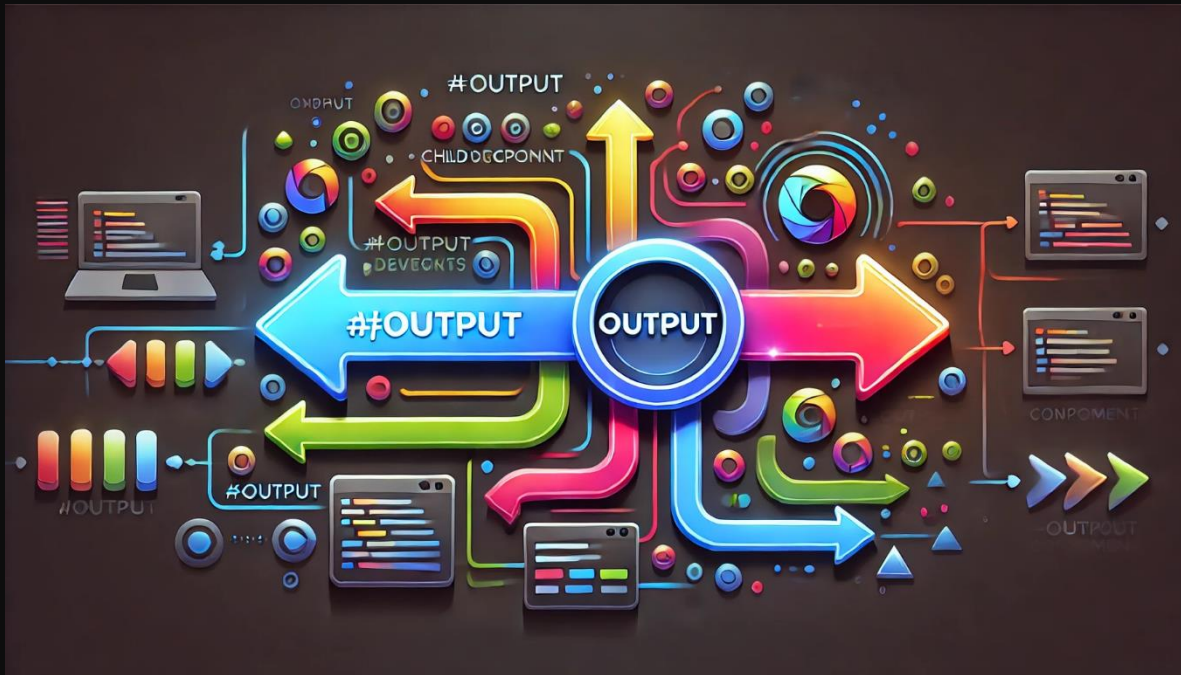


Decorador @Output en Angular

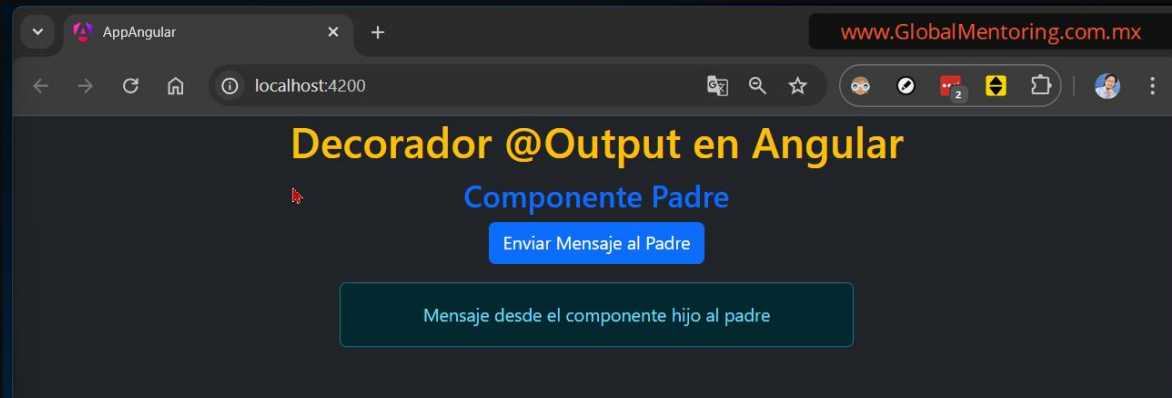


Uso de @Output en Angular

En esta lección, aprenderás a usar el decorador @Output en Angular para comunicar datos desde un componente hijo hacia un componente padre mediante la emisión de eventos. Veremos la sintaxis básica y un ejemplo práctico para entender cómo se maneja este tipo de comunicación entre componentes usando Angular.

1. ¿Qué es @Output en Angular?

El decorador @Output en Angular se utiliza para enviar datos o eventos desde un componente hijo hacia su componente padre. Funciona mediante la creación de un **EventEmitter**, que emite un evento cuando ocurre una acción en el componente hijo, y el componente padre puede escuchar este evento y reaccionar ante él.



2. Sintaxis Básica de @Output

1. Importar el Decorador @Output y EventEmitter:

En el componente hijo, importa @Output y EventEmitter desde @angular/core.

```
import { Component, Output, EventEmitter } from '@angular/core';
```

2. Declarar la Propiedad Decorada con @Output:

Define una propiedad en el componente hijo que será un **EventEmitter**. Esta propiedad se decorará con @Output para que el evento pueda ser escuchado en el componente padre.

```
@Output() notificarPadre = new EventEmitter<string>();
```

3. Emitir el Evento:

Cuando se produce una acción en el componente hijo (como un clic), se llama al método `emit` del **EventEmitter** para enviar datos o simplemente notificar al componente padre.

```
this.notificarPadre.emit('Mensaje desde el Hijo');
```

4. Escuchar el Evento en el Componente Padre:

En la plantilla del componente padre, escucha el evento emitido desde el componente hijo con la sintaxis `(evento)="acciónEnElPadre()"`.

```
<app-hijo (notificarPadre)="recibirNotificacion($event)"></app-hijo>
```

3. Ejemplo Completo: Comunicación Hijo-Padre

Supongamos que tenemos un componente hijo que envía un mensaje al componente padre cuando el usuario hace clic en un botón.

Paso 1: Crear el Componente Hijo

El componente hijo emitirá un evento cuando se haga clic en un botón.

Código del Componente Hijo (`hijo.component.html`)

```
<div class="container">
  <button class="btn btn-primary"
    (click)="enviarMensaje()">Enviar Mensaje al Padre</button>
</div>
```

Código del Componente Hijo (`hijo.component.ts`)

```
import { Component, EventEmitter, Output } from '@angular/core';

@Component({
  selector: 'app-hijo',
  standalone: true,
  imports: [],
  templateUrl: './hijo.component.html',
  styleUrls: ['./hijo.component.css']
})
export class HijoComponent {
  @Output() notificarPadre = new EventEmitter<string>();

  enviarMensaje() {
    // Emitir el evento con un mensaje (se emite un str)
    this.notificarPadre.emit('Mensaje desde el Componente Hijo al Padre');
  }
}
```

Paso 2: Crear el Componente Padre

El componente padre reaccionará al evento emitido desde el componente hijo y mostrará el mensaje.

Código del Componente Padre (`padre.component.html`)

```
<div class="container">
  <h3 class="text-primary">Componente Padre</h3>
  <app-hijo (notificarPadre)="recibirNotificacion($event)"></app-hijo>

  @if (mensaje) {
```

```

    <p class="mt-3 alert alert-info w-50 mx-auto">{{ mensaje }}</p>
  }
</div>

```

Código del Componente Padre (padre.component.ts)

```

import { Component } from '@angular/core';
import { HijoComponent } from "../hijo/hijo.component";

@Component({
  selector: 'app-padre',
  standalone: true,
  imports: [HijoComponent],
  templateUrl: './padre.component.html',
  styleUrls: ['./padre.component.css']
})
export class PadreComponent {
  mensaje: string = '';

  // Se emitió un str, es lo que recibimos
  recibirNotificacion(mensaje: string) {
    this.mensaje = mensaje;
  }
}

```

4. Explicación del Ejemplo

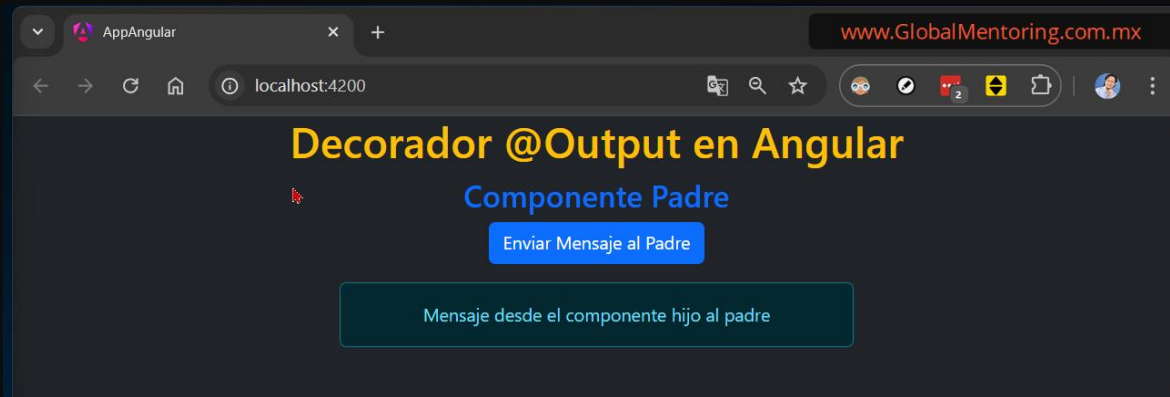
- **Componente Hijo:**
 - Tiene una propiedad `notificarPadre` que es un `EventEmitter<string>`. Este `EventEmitter` emite un evento cuando el método `enviarMensaje()` es llamado (en este caso, cuando el botón es clicado).
 - El evento emitido contiene un mensaje de tipo `string` ('Mensaje desde el Componente Hijo').
- **Componente Padre:**
 - Escucha el evento `notificarPadre` usando la sintaxis `(notificarPadre)="recibirNotificacion($event)"`.
 - El método `recibirNotificacion()` recibe el mensaje enviado desde el componente hijo y lo asigna a la propiedad `mensaje`.
 - El mensaje recibido se muestra en el componente padre mediante una alerta.

5. Puntos Clave para Recordar

- **EventEmitter:** Es la clase que Angular utiliza para emitir eventos personalizados. Cada vez que se llama a su método `emit()`, se desencadena el evento en el componente padre.

- **\$event:** En el componente padre, el parámetro `$event` es el valor que se emite desde el componente hijo. Puedes utilizar este valor para pasar cualquier tipo de dato (string, objeto, número, etc.).
- **Unidireccionalidad:** El flujo de datos va del componente hijo al componente padre mediante la emisión de eventos. Es un flujo de comunicación unidireccional, a diferencia de `@Input`, donde los datos fluyen del padre al hijo.

Resultado Final



Conclusión

El decorador `@Output` es fundamental para la comunicación desde un componente hijo hacia su componente padre en Angular. Entender cómo emitir eventos personalizados y cómo el componente padre puede escuchar y manejar estos eventos es clave para construir aplicaciones Angular modulares y reutilizables.

Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](http://www.globalmentoring.com.mx)